FILE: C:\HCL_TRAINING\client\src\api\client.js

LINES: 107 | SIZE: 2.9 KB | LANGUAGE: JS

MODULE: Resilient Fetch Wrapper with JWT Injection and Error Handlers

```
const API_BASE = '/api';

async function request(endpoint, options = {}) {
  const token = localStorage.getItem('hiresense_token');

  const headers = {
    ...options.headers,
  };

  if (!(options.body instanceof FormData)) {
    headers['Content-Type'] = 'application/json';
  }

  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const config = {
    ...options,
    headers,
    credentials: 'include',
  };

  let response;
  try {
    response = await fetch(`${API_BASE}${endpoint}`, config);
  } catch (networkErr) {
    console.error(`Network fetch failure on ${endpoint}:`, networkErr);
    throw new Error(
      `Cannot connect to API server at ${API_BASE}. Please ensure the backend server is
> running on port 5000.`
    );
  }

  // If unauthorized, clear invalid session tokens
  if (response.status === 401 && !endpoint.includes('/auth/login') && !endpoint.includes
> ('/auth/register')) {
    localStorage.removeItem('hiresense_token');
    localStorage.removeItem('hiresense_user');
  }

  // Handle blob responses (for CSV exports)
  const contentType = response.headers.get('content-type') || '';
  if (contentType.includes('text/csv')) {
    return await response.blob();
  }

  // Handle 204 No Content
  if (response.status === 204) {
    return { success: true };
  }

  // Safely read response as text first
  const responseText = await response.text();

  // If response is completely empty
  if (!responseText || responseText.trim() === '') {
    if (!response.ok) {
      throw new Error(`Server returned error status ${response.status} (${response.statu
> sText || 'Empty Response'})`);
    }
    return { success: true };
  }

  // Parse JSON with defensive fallback
  let data;
```

```
64 |    try {
65 |      data = JSON.parse(responseText);
66 |    } catch (parseError) {
67 |      // If server returned non-JSON (like an HTML error page from a proxy or gateway)
68 |      if (!response.ok) {
69 |        throw new Error(
70 |          `Server returned status ${response.status}: ${responseText.slice(0, 160)}`
71 |        );
72 |      }
73 |      throw new Error(`Unexpected non-JSON response from server: ${responseText.slice(0, 1
   >  60)}`);
74 |    }
75 |
76 |    if (!response.ok) {
77 |      throw new Error(data.message || `Request failed with status ${response.status}`);
78 |    }
79 |
80 |    return data;
81 |  }
82 |
83 |  export const api = {
84 |    get: (endpoint) => request(endpoint, { method: 'GET' }),
85 |    post: (endpoint, body) =>
86 |      request(endpoint, {
87 |        method: 'POST',
88 |        body: body instanceof FormData ? body : JSON.stringify(body || {}),
89 |      }),
90 |    put: (endpoint, body) =>
91 |      request(endpoint, {
92 |        method: 'PUT',
93 |        body: body instanceof FormData ? body : JSON.stringify(body || {}),
94 |      }),
95 |    patch: (endpoint, body) =>
96 |      request(endpoint, {
97 |        method: 'PATCH',
98 |        body: JSON.stringify(body || {}),
99 |      }),
100 |    delete: (endpoint) => request(endpoint, { method: 'DELETE' }),
101 |    upload: (endpoint, formData) =>
102 |      request(endpoint, {
103 |        method: 'POST',
104 |        body: formData,
105 |      }),
106 |  };
107 |
```